كلية الحاسوب والمعلومات
قسم علوم الحاسوب

جامعة أم القرى
UMM AL-QURA UNIVERSITY

CS 1704: البرمجة الشيئية

الفصل الدراسى الثانى، 2026

المحاضرة 2
الجزء 2 - OOP

إعداد:
د. أزهار حسن الهندى

مراجعة: هيكل برنامج جافا الأساسى

- هناك 5 مكونات ندرجها عادة في برنامج جافا، وهي:

```
[ عبارات التوثيق ]
[ عبارة الحزمة
[ عبارات الاستيراد
} public class className
*السمات
*البواني
*الدوال
    public static void main(String[] args) {

        [ جسم الدالة الرئيسية ]
    }
}
```


JAVA: An Introduction to Problem Solving & Programming, 8th Ed. By Walter Savitch
ISBN 0134462033 © 2018 Pearson Education, Inc., Hoboken, NJ. All Rights Reserved

توثيق الصنف الخاص بك

- قبل أن تتمكن من نشر الصنف الجديد -> قم بإعداد توثيق الصنف

- JavaDoc هي أداة توليد توثيق في جافا

- التوثيق باستخدام javadoc -> إنشاء ملف/توثيق HTML تلقائيًا يوفر معلومات حول الأصناف والدوال في المشروع.

- مستند HTML يتم إنشاؤه من التعليقات في برنامج جافا

- تعليقات التوثيق يتم تجاهلها بواسطة مترجم جافا، ولكن تتم معالجتها بواسطة javadoc

- تبدأ بـ /** وتنتهي بـ */

- وسوم تنسيق تبدأ بالرمز @، على سبيل المثال:

@author text

@param parameter-name description

@return description

توثيق الصنف الخاص بك (تابع)

- **ثلاثة مواقع مختلفة في الملف المصدر -> قبل تعريف الصنف العام، الحقل العام أو**

الدالة أو البانى العام.

- لتوليد التوثيق:

٢. في سطر الأوامر -> javadoc filename.java
٤. في Netbeans -> تشغيل -> توليد JavaDoc

توثيق الصنف الخاص بك (تابع)

- مثال:

1 احصل على الصنف Meal واحفظه في ملف آخر يسمى 'MealForJavaDoc.java'.

2 ضع هذا قبل دالة setName (يمكنك إضافة المزيد)

```
/**
* Set the name attribute
* @param n name of meal
*/

public void setName(String n){
name=n;
}
```

```
/**
 * @author Azhar
 */
public class Student {

    String name;

    /**
     *  هذا الطالب.  لـ  الاسم  يغير
     *  عملية قانونية طويلة  قد يتضمن ذلك
     * @param newName This  الطالب  الاسم الجديد.
     *  الاسم الأول يجب أن يتضمن كلاً من
     *  الاسم.  الاسم الأخيرو
     * @return  الاسم الجديد  الاسم الجديد  في  المعامل
     fo *
     public String setName (String newName)
     {
         name = newName;
         return name;
     }
```

جميع الدوال	الدوال الثابتة	دوال المثيل [] دوال ملموسة

المُعدِّل والنوع	الدالة والوصف
static void	main(java.lang.String[] args)
java.lang.String	setName (java.lang.String newName) [] يغير اسم هذا الطالب.

الدوال الموروثة من الصنف [][][][].[][][][].[][][][][][][]

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait,

تفاصيل الباني

Student

public Student ()

تفاصيل الدالة

setName

public java.lang.String setName (java.lang.String newName)

يغير اسم هذا الطالب. قد يتضمن ذلك عملية قانونية طويلة.

المعاملات:

newName [] الاسم الجديد لهذا الطالب. يجب أن يتضمن كلاً من الاسم الأول والاسم الأخير.

المرتجعات:

الاسم الجديد في المعامل

main

هيكل الدالة

o تتكون الدالة من جزأين:

رأس الدالة وجسمها

تعريف دالة

رأس الدالة

المُعدِّل

نوع القيمة المرتجعة

اسم الدالة

المعامل الرسمى

```
public static int max(int
{
    int result;

    if (num1 >      num2 )
        result =    num1;
    else
        result =    num2;

return result;
}
```

num1, int num2)

المعامل

جسم الدالة

توقيع الدالة

return

استدعاء دالة

```
int z = max(i, j);
```

المعاملات الفعلية (الوسائط)

استدعاء دالة يعادل تنفيذ جسم الدالة

تمرير المعاملات

- عند تمرير كائن/مرجع المتغير إلى دالة كمعامل، يتم إرسال مرجعه مما يعني أن أي تغيير على الكائن في الدالة المُستدعاة سينعكس على الكائن الأصلي في الدوال المُستدعية.

- يُسمى هذا "استدعاء بالمرجع" بينما استخدام أنواع البيانات البدائية يُسمى "استدعاء بالقيمة" لأن القيمة الفعلية (نسخة من المتغير) تُرسل إلى الدالة المُستدعاة وبالتالي أي تغييرات لا تؤثر على المتغير الأصلي.

تمرير المعاملات

- استدعاء بالقيمة:

- المعاملات الفعلية يمكن أن تكون حقولاً أو قيماً حرفية أو تعبيرات.
- تعديل المعامل الصوري لا يغير المعامل الفعلي

```
double discount =1.00;
int cal= 400;
Meal salad = new Meal ("Salad",3.99-discount, cal, Meal.VEGETERIAN);


public void Meal (String n, double p, int c , int d){
name = n;
price = p;
c = (int)(c*0.5);
calories = c;
dite = d;
}
```

تمرير المعاملات: مثال

```
public class Main {
   public static void main(String[] args) {
      A a = new A(10);
      int y = 5;
      changeObject(a,y);
      System.out.println(a.x);
      System.out.println(y);
   }

   public static void changeObject( (A b, int z)  {

    b.x =100 Z=50

   }
}
```

```
public class A{
public int x;

public A(int
x){ this.x = x; }
}
```

```
Output:
100
5
```

مرجع مجهول / كائن مجهول

```
Public classA{
Show(){....}
}
```

- Int x=9; //x متغير بدائى
- Aobj; //obj متغير مرجعي
- لا يمكننا استدعاء الدالة show باستخدام obj.
- نحتاج لإنشاء ذلك الكائن أولاً..

Aobj = new A();

- الآن يمكننا الوصول إلى الدالة show:

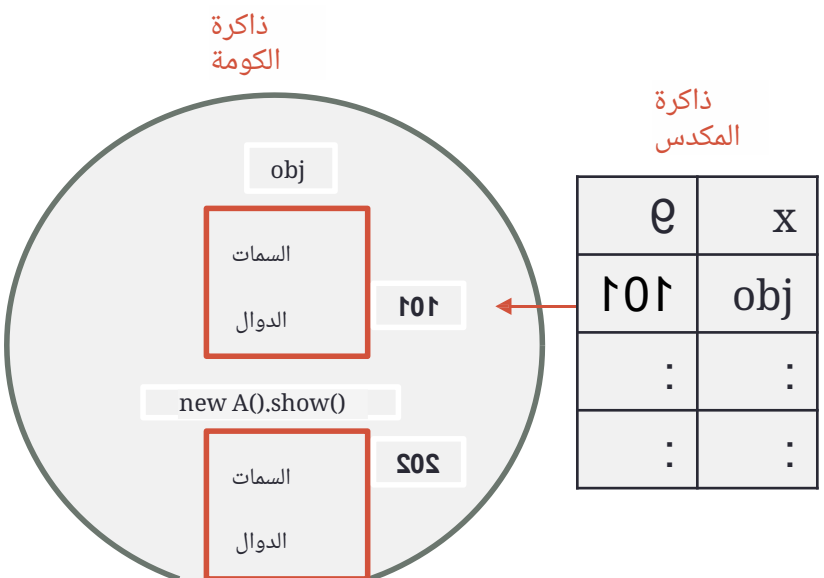
obj.show();

- **إذا احتجنا استخدام هذا الكائن مرة واحدة، لا نحتاج لإنشاء أي كائن مرجعي هنا:**

لذا يمكننا استدعاء الدالة بهذه الطريقة:

new A().show();

- هذا لن يستهلك أي ذاكرة مكدس بل سيستهلك فقط ذاكرة كومة
- لذا، هذا النوع من الكائنات التي ليس لها مرجع/اسم يُسمى كائنًا مجهولًا.



- **مزايا ذلك هي:**
  - لن يستخدم أى ذاكرة داخل المكدس (لتوفير الذاكرة).
  - هذا النوع من الكائنات يُنشأ ويموت فورًا.

علاقات الصفوف

الارتباط
التجميع - التركيب
الوراثة

علاقات الصفوف

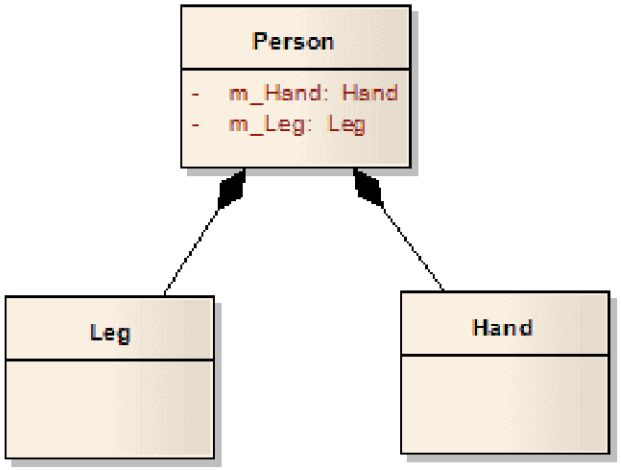
- لـ **تصميم الصفوف،** تحتاج إلى استكشاف العلاقات بين الصفوف.

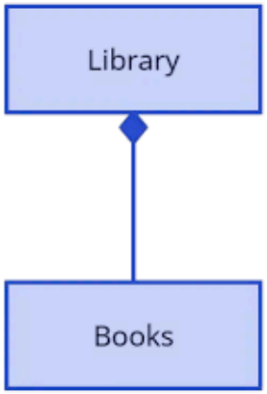
العلاقات الشائعة بين الصفوف هي الارتباط، التجميع، التركيب، والوراثة.

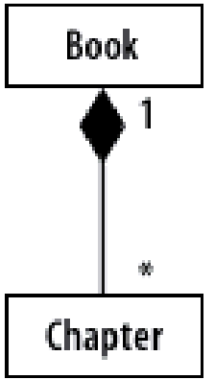
- **الارتباط**
- **التجميع، التركيب**
- **الوراثة (المحاضرة القادمة)**

التركيب الكائني

- التركيب هو طريقة لتصميم أو تنفيذ علاقة يمتلك.
- علاقة يمتلك تُستخدم لضمان إعادة استخدام الكود في برنامجنا.
- في التركيب، نستخدم متغير نسخة يشير إلى كائن آخر.
- علاقة التركيب بين كائنين ممكنة عندما يحتوي كائن على كائن آخر، ويكون ذلك الكائن معتمدًا عليه







The composition relations are implemented using data fields in classes.

مثال

```
public class ADate {
  public static final int JANUARY   = 1;
  public static final int FEBRUARY  = 2;
  public static final int MARCH     = 3;
  public static final int APRIL     = 4;
  public static final int MAY       = 5;
  public static final int JUNE      = 6;
  public static final int JULY      = 7;
  public static final int AUGUST    = 8;
  public static final int SEPTEMBER = 9;
  public static final int OCTOBER   = 10;
  public static final int NOVEMBER  = 11;
  public static final int DECEMBER  = 12;
  private int day, month, year;

  public ADate() {
    day = 0; month = 0; year = 0;
  }
  public ADate(int day, int month, int year) {
    this.day = day; this.month = month; this.year = year;
  }
```

continue

مثال

```
  public ADate copy() {
    ADate d = new ADate();
    d.day = day;
    d.month = month;
    d.year = year;
    return(d);
  }

  public void setDay(int d) { day = d; }
  public void setMonth(int m) { month = m; }
  public void setYear(int y) { year = y; }
  public int getDay() { return(day); }
  public int getMonth() { return(month); }
  public int getYear() { return(year); }

  public String toString() {
    return("(" + day + ", " + month + ", " + year + ")");
  }
}
```

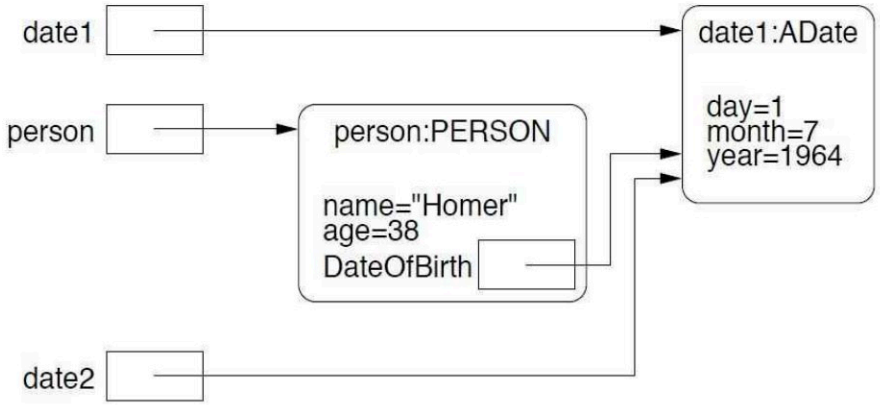
التركيب الكائني

```
public class TestAPerson1 {
  public static void main (String[] args) {
    ADate date1 = new ADate(1, 7, 1964);
    Person person = new Person("Homer", 38, date1);
    System.out.println("person:  " + person);
    ADate date2 = person.getDateOfBirth();
    System.out.println("date2:  " + date2);
  }
}
```

```
public class Person {
  private static final String NO_NAME = "NONAME";
  private String name;
  private int age;
  private ADate dateOfBirth;

  public Person() {
    name = NO_NAME; age = 0; dateOfBirth = new ADate();
  }
  public Person(String n, int a) {
    name = n; age = a; dateOfBirth = new ADate();
  }
  public Person(String n, int a, ADate d) {
    this(n, a); dateOfBirth = d;
  }

    code fragment 1        // mutator and accessor methods

  public String toString () {
    return("(" + name + ", " + age + ", " + dateOfBirth + ")");
  }
}
```

com6050Java and UML fo

Object composition — objects within objects:



التركيب الكائنى

• إنشاء نسخة من الكائنات:

➢النسخ السطحى:

• يتم إنشاء كائن جديد يحتوي على نسخة طبق الأصل من القيم في الكائن الأصلي.

• يتم نسخ عناوين المرجع فقط

➢النسخ العميق:

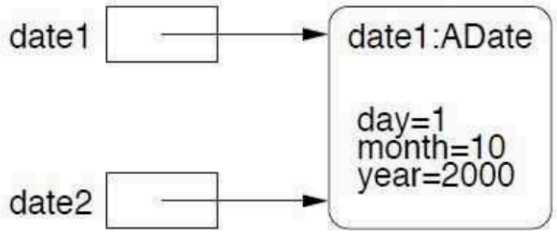
• النسخ العميق ينسخ جميع الحقول ويصنع نسخًا من الذاكرة المخصصة ديناميكيًا المشار إليها بواسطة الحقول.

التركيب الكائني

```
public class TestADate2 {
  public static void main (String[] args) {
    ADate date1 = new ADate(1, 10, 2000);
    ADate date2 = date1;                    // copying the reference
    System.out.println("date1:  " + date1);
    System.out.println("date2:  " + date2);

    System.out.println("Statement:  date2.setDay(3);");
    date2.setDay(3);
    System.out.println("date1:  " + date1);
    System.out.println("date2:  " + date2);
  }
}
```

النسخ السطحي لكائن سيحتوي على نسخة طبق الأصل من جميع حقول الكائن الأصلي. إنه نسخ للمرجع.

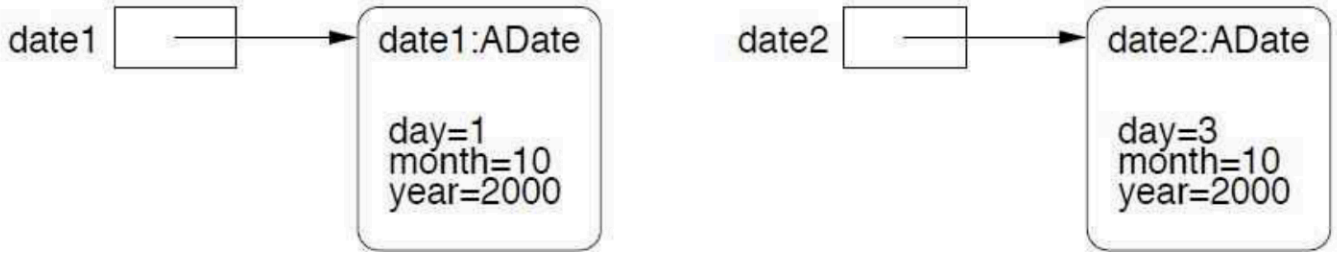
Shallow copy:



التركيب الكائني

```
public class TestADate3 {
  public static void main (String[] args) {
    ADate date1 = new ADate(1, 10, 2000);
    ADate date2 = date1.copy();              // copying the object
    System.out.println("date1:  " + date1);
    System.out.println("date2:  " + date2);

    System.out.println("Statement:  date2.setDay(3);");
    date2.setDay(3);
    System.out.println("date1:  " + date1);
    System.out.println("date2:  " + date2);
  }
}
```

Deep copy:



التركيب الكائنى

- تجنب النسخ السطحي - عرضة للخطأ
- استخدم نسخًا عميقًا.

} (public Person(String n, int a, ADate d
;(this(n, a
;()dateOfBirth = d.copy

```
public void setDateOfBirth(ADate d) {
    dateOfBirth = d.copy();
}
public Date getDateOfBirth() {
    return(dateOfBirth.copy());
```

{

- يمكن أيضًا استخدام مرجع مجهول

;((Person person = new Person("Homer", 38, new ADate(1, 7, 1964

التركيب الكائنى

- يمكن إخفاء الصنف Date داخل الصنف Person

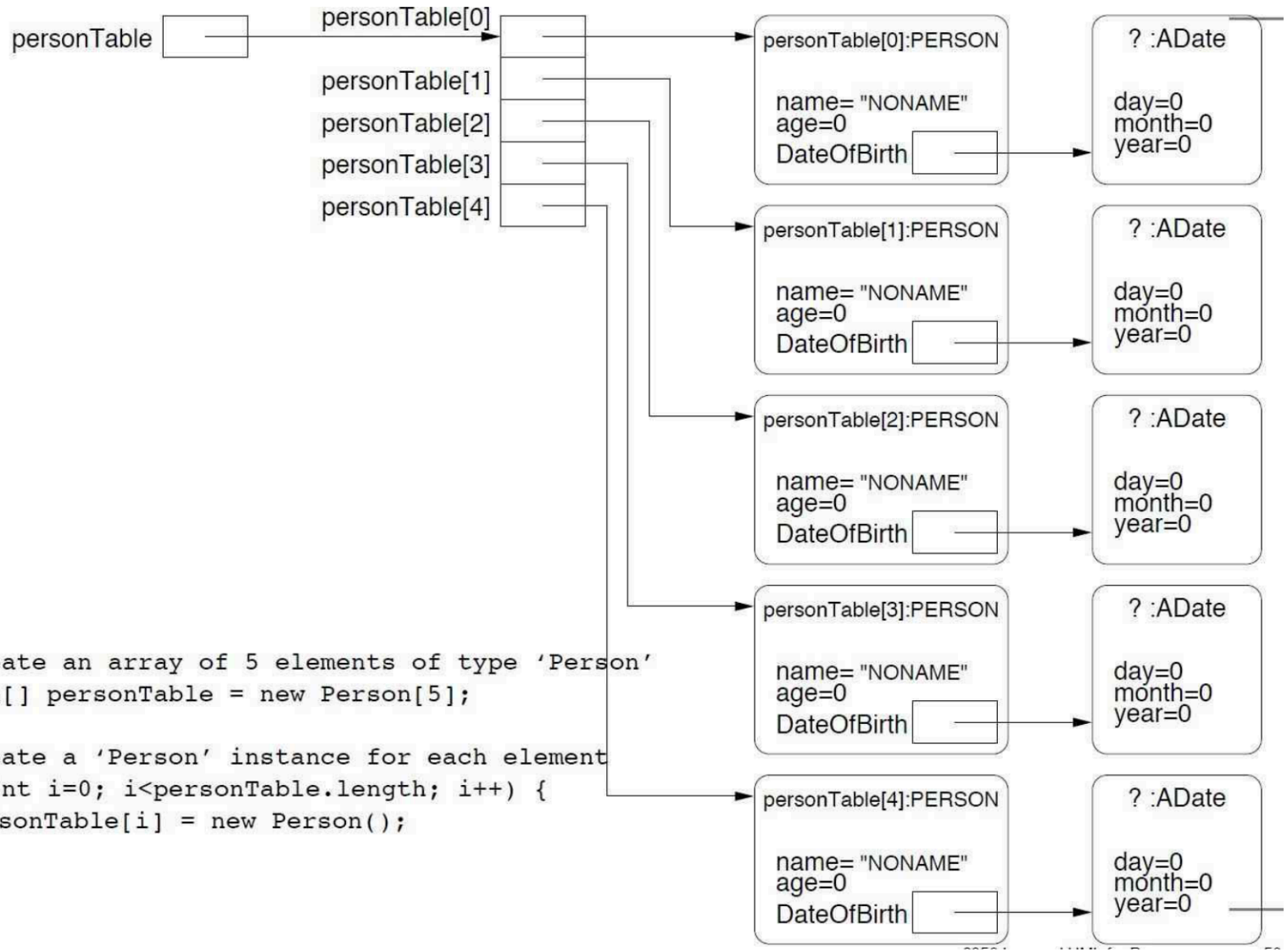
```
Public Person(String n, int a, int d, int m, int y) {
        name= n;
        age = a;
        dateOfBirth = new ADate(d,m,y);
}
public void setDateOfBirth(int d, int m, int y) {
        dateOfBirth = new ADate(d,m,y);
}
```

التركيب الكائنى: مصفوفات الكائنات

arrays of objects can be created in the same way as arrays of Java basic types

```
int[] number = new int[5];              // array of 5 integers
Person[] personTable = new Person[5]; // array of 5 people
```

التركيب الكائني: مصفوفات الكائنات



1
```
// create an array of 5 elements of type 'Person'
Person[] personTable = new Person[5];
```

2
```
// create a 'Person' instance for each element
for (int i=0; i<personTable.length; i++) {
   personTable[i] = new Person();
}
```

المراجع

- الشرائح معدلة من: محاضرات د. عاطف الهجالي، محاضرات د. سيرين، محاضرات د. منال ومراجع أخرى
- https://www.youtube.com/watch?v=qP9-3LnMZsE
- https://www.youtube.com/watch?v=pTB0EiLXUC8&t=75s